

Equivalence Class Testing

Equivalence Class Testing (ECT) is a software testing technique that divides input data into **equivalence classes** or groups. The idea is that if one test case in a group (class) passes or fails, all other values in the same group will behave similarly. This reduces the number of test cases needed while still ensuring effective coverage.

In simpler terms, instead of testing every single possible input, we group inputs into classes and select one value from each class for testing.

Key Concepts:

1. **Equivalence Classes:** These are groups of inputs that are expected to be treated the same by the software. There are two types:
 - **Valid Equivalence Classes:** Inputs that are within the acceptable range.
 - **Invalid Equivalence Classes:** Inputs that are outside the acceptable range.
2. **Goal of Equivalence Class Testing:** Reduce the number of test cases while still ensuring that the software is tested thoroughly.
3. **How to Use ECT:**
 - Identify the valid and invalid input ranges.
 - Divide the inputs into classes.
 - Select one representative test case from each equivalence class.
4. **Benefits:**
 - Reduces the number of test cases.
 - Ensures comprehensive coverage of inputs.
 - Makes testing more efficient.

Steps for Equivalence Class Testing:

1. **Identify the input domain:** Determine the possible input values for the system or function.
 2. **Divide the input domain into equivalence classes:** These classes include both valid and invalid inputs.
 3. **Select test cases:** Pick one representative value from each equivalence class to test the system.
-

Types of Equivalence Classes:

1. **Valid Equivalence Class:** These are the valid input ranges.
 - For example, for a system that accepts ages between 18 and 60, the valid class is 18 to 60.
 2. **Invalid Equivalence Class:** These are the values that are outside the valid range.
 - For the same age example, invalid classes would be ages below 18 and ages above 60.
-

Example 1: Age Validation (Valid Range: 18 to 60)

- **Valid Equivalence Class:** Age values between 18 and 60 (inclusive).
 - Example: Age = 25 (valid)
- **Invalid Equivalence Classes:**
 - **Below the Lower Bound:** Ages less than 18 (invalid)
 - Example: Age = 15 (invalid)
 - **Above the Upper Bound:** Ages greater than 60 (invalid)
 - Example: Age = 70 (invalid)

Test cases:

- Valid test case: Age = 25 (valid)
 - Invalid test case (below boundary): Age = 15 (invalid)
 - Invalid test case (above boundary): Age = 70 (invalid)
-

Example 2: PIN Code Validation (Valid Range: 4 digits, 1000 to 9999)

- **Valid Equivalence Class:** PIN codes between 1000 and 9999.
 - Example: PIN = 1234 (valid)
- **Invalid Equivalence Classes:**
 - **Below the Lower Bound:** PIN codes less than 1000 (invalid)
 - Example: PIN = 999 (invalid)
 - **Above the Upper Bound:** PIN codes greater than 9999 (invalid)
 - Example: PIN = 10000 (invalid)
 - **Length is incorrect:** Any PIN code that doesn't have exactly 4 digits (invalid)
 - Example: PIN = 12 (invalid), PIN = 12345 (invalid)

Test cases:

- Valid test case: PIN = 1234 (valid)
 - Invalid test case (below boundary): PIN = 999 (invalid)
 - Invalid test case (above boundary): PIN = 10000 (invalid)
 - Invalid test case (wrong length): PIN = 12 (invalid)
-

Example 3: Temperature Input Validation (Valid Range: -10°C to 50°C)

- **Valid Equivalence Class:** Temperatures between -10°C and 50°C.
 - Example: Temperature = 20°C (valid)
- **Invalid Equivalence Classes:**

- **Below the Lower Bound:** Temperatures less than -10°C.
 - Example: Temperature = -20°C (invalid)
- **Above the Upper Bound:** Temperatures greater than 50°C.
 - Example: Temperature = 60°C (invalid)

Test cases:

- Valid test case: Temperature = 20°C (valid)
 - Invalid test case (below boundary): Temperature = -20°C (invalid)
 - Invalid test case (above boundary): Temperature = 60°C (invalid)
-

Scenario-Based Exam Questions on Equivalence Class Testing

1. **Scenario:** A system accepts a user's age as input and only allows ages between 18 and 100 (inclusive). Write test cases using Equivalence Class Testing.

Answer:

- **Valid Equivalence Class:** Ages between 18 and 100 (e.g., 25, 50, 75).
- **Invalid Equivalence Classes:**
 - Below the lower bound: Age = 15 (invalid)
 - Above the upper bound: Age = 105 (invalid)

Test cases:

- Test Case 1: Age = 25 (valid)
 - Test Case 2: Age = 15 (invalid, below boundary)
 - Test Case 3: Age = 105 (invalid, above boundary)
-

2. **Scenario:** A website requires users to enter their username with a length of between 6 and 12 characters (inclusive). Create the equivalence classes and corresponding test cases.

Answer:

- **Valid Equivalence Class:** Username lengths between 6 and 12 characters.
- **Invalid Equivalence Classes:**
 - Below the lower bound: Username = "abc" (invalid)
 - Above the upper bound: Username = "abcdefghijkl" (invalid)

Test cases:

- Test Case 1: Username = "abcdef" (valid, 6 characters)
 - Test Case 2: Username = "abcdefghijk" (valid, 11 characters)
 - Test Case 3: Username = "abc" (invalid, below boundary)
 - Test Case 4: Username = "abcdefghijkl" (invalid, above boundary)
-

3. **Scenario:** You are testing a password system that accepts passwords with 8 to 16 characters. Use Equivalence Class Testing to generate test cases.

Answer:

- **Valid Equivalence Class:** Password lengths between 8 and 16 characters.
- **Invalid Equivalence Classes:**
 - Below the lower bound: Password = "abc" (invalid)
 - Above the upper bound: Password = "aVeryLongPassword" (invalid)

Test cases:

- Test Case 1: Password = "abcdefgh" (valid, 8 characters)
- Test Case 2: Password = "abcdefghijklm" (valid, 13 characters)
- Test Case 3: Password = "abc" (invalid, below boundary)

- Test Case 4: Password = "aVeryLongPassword" (invalid, above boundary)
-

4. **Scenario:** A form asks users to enter their credit card number, which must be exactly 16 digits long. Create the equivalence classes for this scenario and write test cases.

Answer:

- **Valid Equivalence Class:** Credit card numbers exactly 16 digits long.
- **Invalid Equivalence Classes:**
 - Below the lower bound: Credit card number = "123456789012" (invalid, 12 digits)
 - Above the upper bound: Credit card number = "12345678901234567" (invalid, 17 digits)

Test cases:

- Test Case 1: Credit card number = "1234567890123456" (valid, 16 digits)
 - Test Case 2: Credit card number = "123456789012" (invalid, below boundary)
 - Test Case 3: Credit card number = "12345678901234567" (invalid, above boundary)
-

Equivalence Class Testing helps reduce the number of test cases by grouping inputs into classes, assuming that testing one value from each class is enough to verify the system's behavior for that class. It is a highly effective technique for ensuring efficient and thorough testing, especially when dealing with large input domains. By selecting test cases from valid and invalid equivalence classes, you can test the software while minimizing redundancy.
